

AutoBOM — Product Discovery & Analysis Report

Prepared by: Lead Product Architect / Systems Architect **Phase:** Discovery (pre-implementation) **Status:** Draft for stakeholder review **Inputs reviewed:** Product Vision & Scope document; six workflow diagrams (current workflow, system workflow V1, decision engine V1, and the Designer / Production / Purchasing role flows); two stakeholder conversation transcripts ("Convo with CTO" P1 & P2); the proof-of-concept codebase (Streamlit app, CLI, BOM cleaner, sourcing engine, supplier matcher, Mouser / DigiKey / Nexar / PartsBox clients, caching, exporters); and two sample BOM files.

A note on method. This report is deliberately a *discovery* document, not an implementation plan. It challenges assumptions where the evidence warrants it, and it is explicit about the difference between what the proof-of-concept (POC) actually does today and what the vision describes. The single most important finding, stated up front: **the POC is a single-user batch automation script, and the vision is a multi-user, stateful, collaborative workflow platform. Those are different kinds of software.** Most of the recommendations flow from that gap.

Section 1 — Product Understanding

What AutoBOM is

AutoBOM is an internal engineering operations platform for a hardware company that designs and builds electronic units. Its job is to carry an electronic design from the moment a part is first considered, through sourcing and purchasing, into inventory — and to keep every department working from the same record instead of from email, spreadsheets, Teams messages, and memory.

Today, the working software is narrower than that vision. The README describes it accurately: an *"internal BOM automation tool for cleaning designer BOMs, creating PartsBox project setup, sourcing parts through Mouser/DigiKey, and preparing supplier carts/lists."* It is a file-in / file-out batch tool. The vision in the scope document and the conversations is considerably larger: a system of record connecting Design, Production, Purchasing, and (later) Inventory and Management.

What business problems it solves

The conversations describe a set of concrete, recurring pains that the platform exists to remove:

- **Slow, manual sourcing.** The current-workflow diagram puts the end-to-end manual process at roughly **3.5-4.5 hours per BOM**, with "Search Mouser / Search DigiKey"

alone taking 1h20-2h and supplier comparison another hour. The cleaning step (10-30 min of fixing manufacturer names and part numbers) is also manual today.

- **Messy designer BOMs.** Real exports (see `test_bom2.csv`), an Altium-style export) arrive with encoding noise ($\oplus$ rendered as $\oplus$), combined manufacturer/part-number fields, inconsistent manufacturer names, multi-designator rows, and junk columns. Someone has to clean these by hand before anything can be ordered.
- **Parts going obsolete between design and build.** A part that was in stock when the design started is frequently out of stock or end-of-life two to three months later when production actually orders it. There is no systematic way to detect this early or to find replacements.
- **No handoff system between people.** Work moves between the designer, the person doing sourcing, and the purchaser by talking, emailing, and remembering. Requests get stuck in "limbo" while one person waits on another.
- **Order and receiving chaos.** Orders for a project, for wall (stock) replenishment, and for individual engineers get mixed together. When parts arrive, nobody can tell whose part is whose. People repeatedly ask "did you order it? has it shipped? which order was that?" because order status and tracking are not visible in one place.
- **No traceability.** There is no durable answer to "who owns this, what project is it for, where did it come from, what is its status."

Why it exists

Because the company is amplifying a workflow that currently lives in one or two people's heads and hands. The founder/lead is explicit that the goal is not merely "fewer clicks" — it is to make the work *faster, more intuitive, less error-prone, and able to move between people without anyone having to chase it*. The deeper motivation is to turn an informal, person-dependent process into a system that scales as the team, the project count, and the supplier/inventory footprint grow.

Who the users are

From the conversations, real named roles emerge (names paraphrased):

- **Designers / R&D engineers** — look up parts *while designing*, build informal lists of candidate parts, and decide designs partly based on availability. They are the origin of every part.
- **Production / Sourcing** — receive a finished BOM, validate and clean it, source it, and prepare it for purchase. This is where the current POC is most useful today.
- **Purchasing** — review requests and actually place orders on Mouser/DigiKey, then track shipments.
- **Inventory / Receiving** (future) — receive shipments, sort parts by owner/project, and decide what goes "to the wall" (stock) versus to a project versus to scrap.

- **Management** (future) — oversight, approvals, and reporting.

The team is small. The conversations imply a handful of named users, a part-time build cadence (30–45 minutes a day), and a developer who is primarily a hardware/design person learning to build software. This matters enormously for architecture and roadmap decisions later in this report.

How departments interact

The intended interaction is a **directional flow with push/pull handoffs**, explicitly modeled on issue-tracking and GitHub-style workflows:

1. A **designer** collects candidate parts, then either parks the list or *requests an order*.
2. **Production** validates and sources a BOM; if parts are unavailable it *pushes the BOM back to the designer* with an exception report.
3. The **designer** supplies replacements and *pushes back* to production.
4. Production submits a procurement package to **purchasing**.
5. **Purchasing** reviews, approves or rejects (with comments), places the order, and attaches tracking.
6. **Inventory** (future) receives, attributes parts to owners/projects, and updates stock.

The key design intent is that *work moves through the system* rather than depending on people remembering conversations, and that each step generates a notification so the next person knows to act — while the previous person is freed from waiting in limbo.

What success looks like

- A designer can check sourcing and build a named parts list in minutes, without touching Mouser/DigiKey directly.
 - A BOM goes from "received" to "ready for purchasing" with cleaning, sourcing, and exception-handling largely automated and the human stepping in only to approve and to choose replacements.
 - Anyone can answer "who owns this / what project / what's its status / where's the shipment" by looking at the platform, not by asking around.
 - Critical actions (placing orders, approving changes) always pass through a human, but everything leading up to them is prepared automatically.
 - The team adopts it because it is genuinely faster than the status quo — not because they are told to.
-

Section 2 — Workflow Analysis

Existing workflows (as built in the POC)

The POC implements a single linear batch pipeline, visible in `main.py` and the Streamlit `app.py`, and matching the **current-workflow** and **system-workflow-V1** diagrams:

1. **Upload / select BOM** (CSV or XLSX).
2. **Clean & normalize** — header normalization, junk-column and empty-row removal, field mapping, manufacturer/MPN canonicalization, designator counting, duplicate-part consolidation (`bom_cleaner.py`).
3. **Apply build quantity** — $\text{required_qty} = \text{qty_per_board} \times \text{build_quantity}$ (`apply_project_quantities`).
4. **Source** — Mouser-first, DigiKey-fallback supplier lookup with caching (`sourcing_engine.py`, supplier clients).
5. **Decide supplier** — the decision engine selects a single supplier that can cover the full quantity, or flags for review (`decide_no_split_supplier`).
6. **Generate outputs** — cleaned BOM workbook/CSV, sourcing report, DigiKey list CSV, optional Mouser cart, optional DigiKey MyList, PartsBox project + entries, run-summary JSON.

The **decision-engine-V1** diagram matches the code almost exactly: normalized BOM → search → collect supplier data (stock, price, availability, lead time) → compare → *Can Mouser fulfill quantity?* → if not, *Can DigiKey fulfill quantity?* → else *flag for engineering review*.

Missing workflows (in the vision but not built)

- **Designer collection workflow** — creating, editing, and persisting named personal/project parts lists ("collections") that are *not* BOMs and *not* yet orders. This is the headline feature request in conversation P1, and it does not exist in code at all. There is no persistence layer to hold it.
- **Push-back / exception workflow** — production returning a BOM to the designer with an exception report, the designer supplying replacements, and the BOM returning to production. The diagrams model it; the code does not implement any handoff.
- **Purchasing review/approve/reject workflow** — the purchasing role diagram (receive request → notify → approve? → create PO / send back with comments → update status) is entirely unbuilt. Ordering today is intentionally manual and done directly on supplier sites.
- **Order status & tracking** — pending → pending review → ordered → shipped (with tracking) → received. Wanted repeatedly in P2; not built.

- **Replacement / alternate suggestion** — "take the characteristics of the part and give me five alternates of each, and I pick one." This is the most technically ambitious request and is not implemented. (The dormant Nexar/Octopart client is the natural foundation for it — see Section 4.)
- **Inventory / "the wall"** — receiving, attribution of parts to owner/project, the "to the wall vs. to a project vs. scrap" decision, and stock levels. Referenced constantly; entirely absent from code. A `check_wall_inventory` status string exists, but there is no wall data behind it.
- **Notifications** — in-app notification feed, email, or Teams integration. Acknowledged as the hard part; not built.
- **Identity, roles, and role-specific UI** — the vision wants a role picker / per-role front ends ("Raul's corner"). The POC has no users, no auth, and one generic UI.

Incomplete workflows

- **Sourcing** stops at "Mouser or DigiKey can cover the full quantity." It does not handle split orders, partial coverage, price optimization across suppliers, lead-time weighting (despite "Lead Time" being listed as collected data in the decision-engine diagram), or third suppliers. Notably, `test_bom2.csv` already contains **Newark** as a supplier — a supplier the engine does not handle.
- **Build overage** — the production role diagram lists "Overage %" as part of normalization and the procurement package. The code computes $\frac{\text{qty_per_board} \times \text{build_quantity}}$ with no overage factor.
- **PartsBox** integration creates projects/storage and adds entries, but its relationship to the future "inventory" module and "the wall" is undefined. It may be the inventory system of record, or it may be replaced.

Workflow bottlenecks

- **Supplier API latency is the structural bottleneck**, not anything in the team's own logic. The code enforces a Mouser search delay (default 2.1s) and rate-limit retries; DigiKey uses fragile 3-legged OAuth. A large BOM sourced sequentially is inherently slow. This is an I/O-bound problem and shapes the architecture (background jobs + caching, not synchronous requests).
- **Manual cleaning** of messy exports is partly automated but still produces "manual review" rows that a human must resolve.
- **The handoff gaps** — every transition between people is currently a manual conversation, which is the core inefficiency the platform is meant to remove.

Opportunities for automation

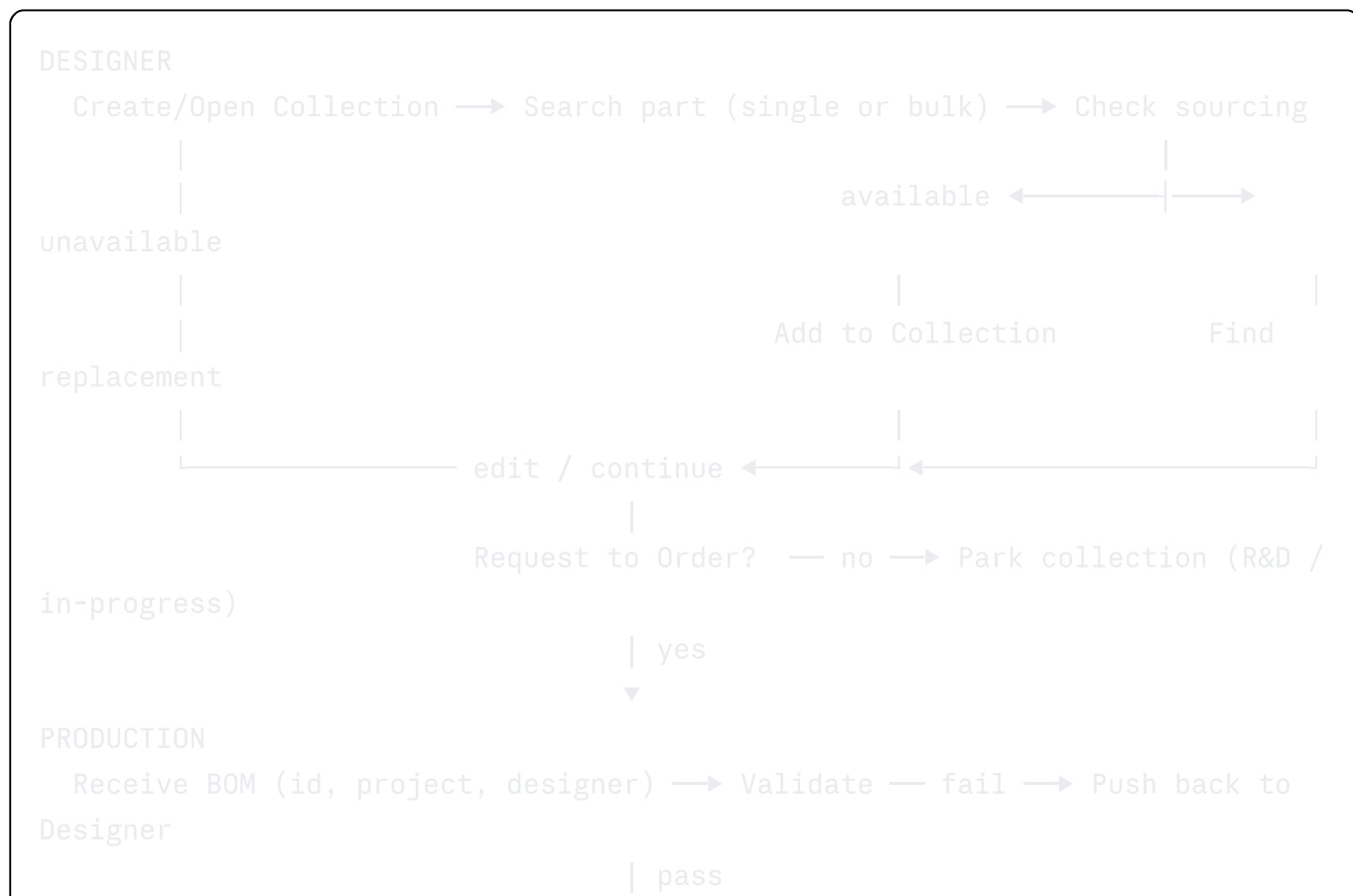
- Auto-clean and auto-normalize on upload (already strong; extend it).

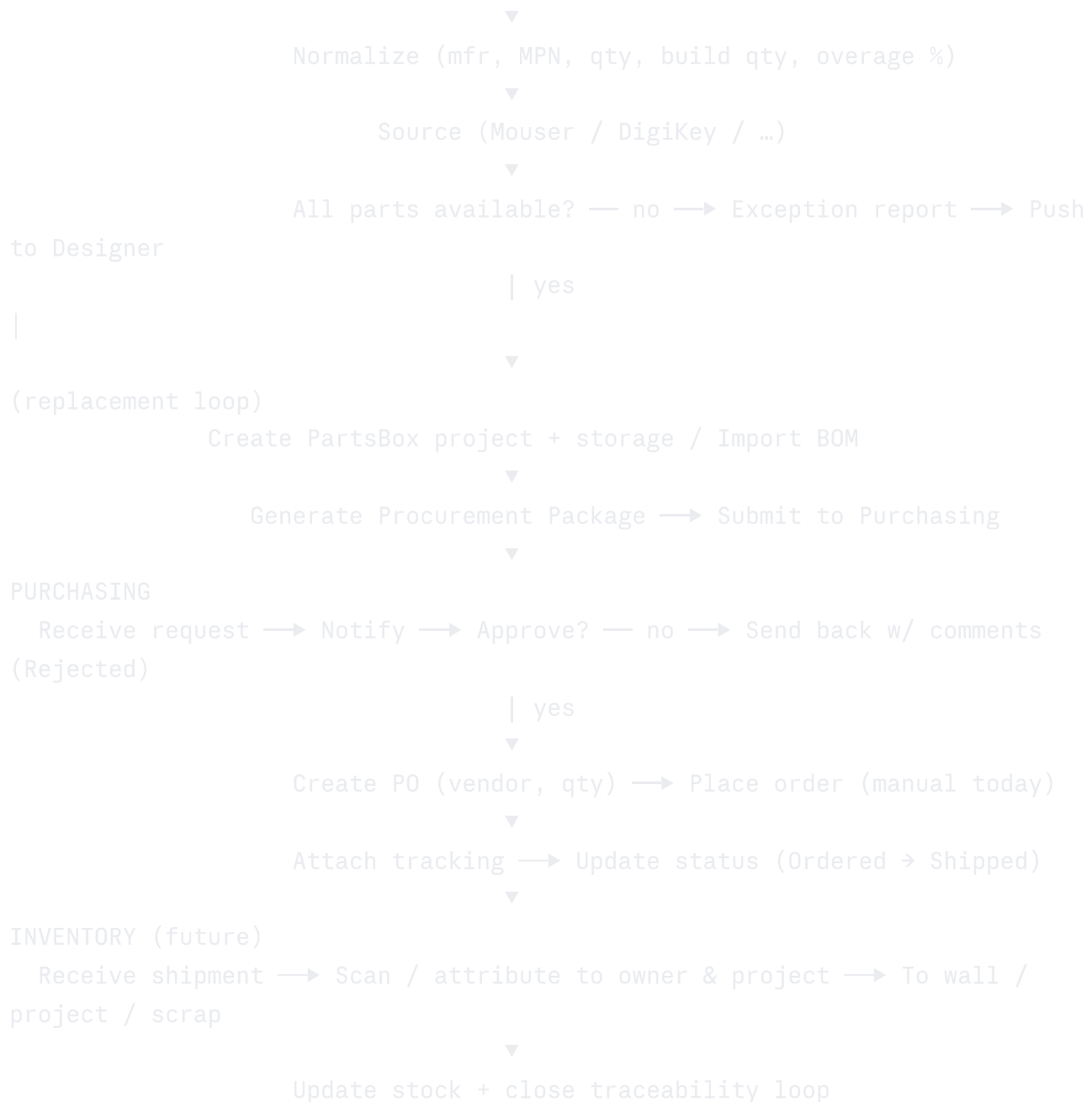
- Cache and reuse sourcing results aggressively (already started via `SupplierLookupCache`; make it a first-class, shared, time-aware cache).
- Auto-generate the procurement package and pre-fill purchasing carts (prepare, never auto-submit).
- Auto-detect newly-obsolete parts by re-sourcing a stored BOM and diffing against the last result.
- Auto-attribute orders via a customer-reference field round-tripped through the supplier, or via barcode scanning at receiving.

Opportunities for simplification

- Replace the multi-toggle Streamlit screen (Run PartsBox / Run sourcing / Use Mouser / Use DigiKey / Create cart / Create MyList) with role-appropriate defaults so users aren't configuring a pipeline on every run.
- Add the **manual part-entry search bar** the founder explicitly flagged as missing ("right now it's stuck at upload a file"). Single-part lookup is a frequent need and currently impossible.
- Collapse the "make a Mouser cart / DigiKey MyList just to save parts" workaround into a native, persisted collection — removing the dependence on supplier-side list features.

End-to-end workflow map (target state)





Every node in this map should emit a status change and (where it crosses a person boundary) a notification. That cross-boundary eventing is the platform's reason for existing.

Section 3 — User Role Analysis

Designer

- **Goals:** evaluate part availability while designing; keep personal/project candidate lists; hand off an order request without losing context.
- **Responsibilities:** originate parts and BOMs; respond to exception push-backs with replacements; remain accountable for parts they ordered until those parts are

dispositioned.

- **Inputs:** part numbers / search terms; design context; obsolete-part notifications from production.
- **Outputs:** named collections; order requests; replacement selections.
- **Pain points:** today they abuse Mouser carts / DigiKey lists as a "save" mechanism; they wait in limbo for sourcing answers; their R&D part orders get lost in the general order flow.
- **Required screens:** part search (single + bulk), collection list view (item count + per-supplier breakdown, e.g. "10 items: 6 Mouser, 4 DigiKey"), collection editor, "request to order," exception/replacement inbox.
- **Required actions:** create/edit/delete collection; check sourcing; add part; find/select replacement; submit order request; respond to push-back.

Production / Sourcing

- **Goals:** turn a finished BOM into a clean, fully-sourced, purchase-ready package quickly; surface problems early.
- **Responsibilities:** validate, normalize, source, handle exceptions, create PartsBox project/storage, generate the procurement package, submit to purchasing.
- **Inputs:** designer BOMs (often messy real-world exports); build quantity and overage; designer replacements.
- **Outputs:** cleaned BOM, sourcing report, exception report, PartsBox project, procurement package.
- **Pain points:** manual cleaning; slow sequential supplier lookups; no structured way to push exceptions back to a designer; parts obsolete since design.
- **Required screens:** BOM upload/validation, normalization review, sourcing results dashboard (sourced / fallback / needs-review), exception management + push-back composer, procurement-package builder.
- **Required actions:** validate; normalize with build qty/overage; run sourcing; flag/exception; push to designer; create PartsBox project; submit to purchasing; set status.

Purchasing

- **Goals:** place the right orders quickly and keep status/tracking visible to everyone.
- **Responsibilities:** review procurement requests; approve/reject with comments; place orders on Mouser/DigiKey; attach tracking; update status.
- **Inputs:** procurement packages; supplier carts/lists; order confirmations and tracking numbers.

- **Outputs:** purchase orders; rejections with required changes; order status + tracking updates.
- **Pain points:** today they manually hunt through supplier order histories to match orders to people and to find tracking; they get repeatedly interrupted with "did you order it / where is it." They (and the founder) explicitly **do not yet trust automated ordering**.
- **Required screens:** request inbox; review screen showing how the order will appear on the supplier site, sortable (e.g., highest price first); PO creation; status/tracking update; rejection composer.
- **Required actions:** approve/reject; create PO; record vendor/tracking/status; send back with comments.

Inventory / Receiving (future)

- **Goals:** receive shipments and attribute every part to the right owner/project with minimal manual sorting; manage "the wall."
- **Responsibilities:** receive, attribute, disposition (wall / project / scrap), maintain stock counts.
- **Inputs:** physical shipments; PO/order references; customer-reference tags or barcode scans.
- **Outputs:** updated inventory; closed traceability from request to received part.
- **Pain points:** the headline receiving pain — *"there are five orders and nobody knows which is which."* Two candidate solutions surfaced: (1) a **customer-reference field** (e.g., "Raul / TVCA / line 1") round-tripped through the supplier and echoed back on the packing data; (2) **barcode scanning middleware** at receiving that maps a scanned part to its order/owner/project before it hits PartsBox.
- **Required screens:** receiving queue, scan/attribution screen, disposition checklist (wall / project / scrap), stock view.
- **Required actions:** receive; scan/attribute; disposition; adjust stock.

Management (future)

- **Goals:** oversight, approvals on high-value or high-risk actions, and reporting.
- **Responsibilities:** approval routing; visibility into cost, cycle time, and bottlenecks.
- **Inputs:** the platform's audit trail and status data.
- **Outputs:** approvals; reports/metrics.
- **Pain points:** none directly stated; this role is implied by the "checks & balances" and "analytics" principles.
- **Required screens:** approval queue; dashboards (spend, cycle time, exception rate, supplier mix).

- **Required actions:** approve/deny; view analytics.
-

Section 4 — Current System Assessment

The POC is a **single-user, local, batch-oriented Python application** (Streamlit UI + CLI), with no database, no authentication, no users, no notifications, and no durable state beyond output files and a JSON lookup cache. Within those limits, several pieces of it are genuinely valuable and should survive into the platform.

A small but telling discrepancy: the README and `app.py` reference a `src/` layout and a `workflow_runner.py` that are **not present in the shared snapshot**. Code organization is informal and flat, modules mix concerns, logging is via `print`, and several `try/except` blocks **catch broadly and continue** (e.g., PartsBox and cart steps print an error and proceed). That last point is worth flagging against the stated reliability principle that "data should never be silently lost / failures should be logged" — the current behavior is closer to "errors are printed and swallowed."

Area	Classification	Why
BOM cleaning & normalization ((bom_cleaner.py))	Keep (refactor into a library)	This is the crown jewel. It handles real Altium exports: header normalization, junk-column/empty-row removal, field mapping via alias detection, encoding-noise repair, designator counting (including ranges like (R1-R3)), combined manufacturer+MPN parsing, and duplicate-part consolidation. Hard-won domain logic; do not rewrite from scratch.
Manufacturer alias + MPN equivalence ((manufacturer_aliases.py))	Keep	Canonicalizes manufacturer names (AVX→Kyocera AVX, ST→STMicroelectronics, Atmel→Microchip, etc.) and normalizes part numbers including scientific-notation artifacts (e.g., capacitor values exported as (1E+05)). Directly improves match accuracy.
Supplier decision engine ((sourcing_engine.py))	Keep core, Expand logic	The Mouser-first / DigiKey-fallback / flag-for-review logic is sound and matches the diagram. But it is single-supplier, full-quantity-only, with no split orders, price optimization, lead-time weighting, or third suppliers. Keep the structure; expand the policy and make the ranking criteria configurable.
Supplier API clients ((mouser_client.py), (digikey_client.py))	Keep, Refactor	Solid matching with exact → relaxed → supplier-part-number → keyword-search fallbacks and candidate ranking. Refactor for async I/O, centralized rate-limit handling, and structured errors instead of print+continue.

Area	Classification	Why
Supplier lookup cache ((supplier_lookup_cache.py))	Expand	Run-level dedup + persistent JSON cache is the right instinct given API rate limits. Promote it to a shared, time-aware cache in the database/Redis with freshness (stock data goes stale fast).
PartsBox integration ((partsbox_client.py), (partsbox_project_builder.py))	Keep, but clarify role	Works (create/reuse project + storage, add entries, export import/unmatched CSVs). Open question: is PartsBox the long-term inventory system of record, or a stopgap to be absorbed by the future Inventory module? This decision affects the data model.
Nexar / Octopart client ((nexar_client.py), (supplier_matcher.py))	Expand / Revive	Currently dormant — the enrichment call is commented out in (main.py). But it provides multi-seller offers, inventory, and price breaks, which is exactly what the replacement/alternate-suggestion feature and multi-supplier optimization will need. Strong candidate to revive as the sourcing engine grows.
DigiKey MyLists / Mouser Cart clients	Refactor / De-emphasize	These exist to work around suppliers' own list/cart features (the founder noted DigiKey lists are named/organized; Mouser carts are just timestamped). Once native collections exist, these become <i>export targets at purchase time</i> , not the primary save mechanism. Keep as integrations, demote as the user-facing "save."
OAuth setup ((digikey_oauth_setup.py))	Refactor	3-legged OAuth with manual refresh-token pasting is fine for a solo POC but unworkable for a multi-user service. Needs server-side credential management.
Streamlit UI ((app.py))	Replace	A single generic screen with six pipeline toggles cannot express role-specific UX,

Area	Classification	Why
		persistence, collaboration, or notifications. Streamlit was the right prototype; it is not the platform front end.
CLI (<code>main.py</code>)	Refactor → orchestration library	The orchestration logic is reusable, but it should become a service/library invoked by an API, not an interactive <code>input()</code> script.
Exporters (<code>sourcing_report.py</code> , <code>digikey_list_export.py</code>)	Keep	Useful output formats; keep them as on-demand exports from the platform.
Persistence / state	Build (does not exist)	No database, no objects, no lifecycle. This is the largest missing foundation.
Identity / auth / roles	Build (does not exist)	No users. The "role picker" vision requires this.
Notifications / workflow engine	Build (does not exist)	The collaboration layer is entirely greenfield.

Bottom line: keep the *domain logic* (cleaning, aliases, decision engine, supplier clients, caching, PartsBox), replace the *delivery mechanism* (Streamlit + CLI + file I/O), and build the *missing foundations* (persistence, identity, workflow, notifications). The POC has proven the hard sourcing logic works in practice — the founder confirmed it correctly sourced and organized real orders. That validation is valuable; preserve it.

Section 5 — Feature Inventory

Existing features (built and working)

- BOM upload (CSV/XLSX) with robust cleaning and normalization.
- Build-quantity math (`required_qty = qty_per_board × build_quantity`).
- Mouser + DigiKey sourcing with multi-strategy matching and candidate ranking.
- Mouser-first / DigiKey-fallback decision engine with "needs review" flagging.
- Supplier lookup caching and in-run deduplication.
- PartsBox project/storage creation and BOM entry import.
- Output generation: cleaned workbook/CSV, sourcing report, DigiKey list CSV, PartsBox import/unmatched CSVs, run-summary JSON.
- Optional Mouser cart and DigiKey MyList creation (toggled, dry-run capable).

- Dry-run modes across PartsBox and supplier integrations.

Planned features (in diagrams/conversations, clearly intended next)

- Designer **collections** (persistent named lists, pre-BOM/pre-order).
- **Single-part search bar** (manual entry, not just file upload). (*Explicitly flagged as missing.*)
- BOM **validation** gate (required columns, part numbers present, valid quantities) with push-back.
- Production **exception report** + **push to designer** + **reprocess** loop.
- Purchasing **review** / **approve** / **reject** with comments.
- **Order status** + **tracking** ingestion (Ordered → Shipped → tracking number).
- **Notifications** (in-app first).
- **Identity, roles, role-specific UI.**

Missing features (needed for the vision, not yet scoped in detail)

- **Replacement** / **alternate-part suggestion** by parametric characteristics.
- **Inventory** / "the wall" module: receiving, attribution, disposition, stock.
- **Order attribution** via customer-reference field and/or barcode scanning.
- **Audit trail** / **change history** / **comment history** on every object.
- **Approval routing** for high-value/high-risk actions.
- **Build overage** % handling.
- **Multi-supplier** / **split-order** / **third-supplier** (e.g., Newark) support.

Future features (longer horizon)

- **Trusted automated ordering** (opt-in, guarded) once confidence is earned.
- **Analytics** / **reporting** (spend, cycle time, exception rate, supplier mix).
- **ERP integration**, multi-location inventory, mobile access, cloud deployment.

Priority grouping

- **P0 — Foundation (must exist before anything else scales):** persistence/system of record; identity + roles; sourcing logic extracted into a reusable service.
- **P1 — Core value:** designer collections; single-part search; production validate→source→exception loop; purchasing review/approve; order status + tracking; in-app notifications.
- **P2 — Differentiators:** replacement/alternate suggestions; inventory + receiving attribution; audit/comment history; overage; multi-supplier.

- **P3 — Advanced:** automated ordering; analytics; ERP/mobile/multi-location.
-

Section 6 — Data Model Analysis

The POC has *no* persisted objects. The platform needs a real domain model. Below are the major objects, with purpose, key relationships, ownership, and lifecycle. (Field lists are illustrative, not final.)

User — *Purpose:* identity and accountability. *Relationships:* has Role(s); owns Collections, BOMs, Requests; is assignee on Notifications/Reviews. *Ownership:* self / admin. *Lifecycle:* invited → active → deactivated.

Role / Permission — *Purpose:* gate features and screens per department (Designer, Production, Purchasing, Inventory, Management). *Relationships:* many-to-many with User. *Lifecycle:* defined → assigned → revised.

Component / Part (catalog) — *Purpose:* canonical record of a part (manufacturer, MPN, description, lifecycle status, parametric attributes). *Relationships:* referenced by CollectionItems, BOMLines, SupplierOffers, InventoryItems. *Ownership:* system. *Lifecycle:* active → obsolete/EOL (drives obsolescence alerts).

SupplierOffer (sourcing result cache) — *Purpose:* a time-stamped result of sourcing a part at a supplier (supplier, supplier PN, stock, price breaks, lead time, URL). *Relationships:* belongs to Component; produced by a sourcing run. *Ownership:* system. *Lifecycle:* fetched → fresh → stale → refreshed. **Freshness matters** — stock data ages fast.

Collection — *Purpose:* a designer's persistent named list of candidate parts (the headline P1 request). *Relationships:* owned by User; belongs optionally to a Project; has many CollectionItems; can spawn a ProcurementRequest. *Ownership:* the designer. *Lifecycle:* draft/in-progress → parked (not ordered) → order-requested → closed.

CollectionItem — *Purpose:* one part in a collection with notes and chosen supplier. *Relationships:* belongs to Collection; references Component. *Lifecycle:* added → edited → removed / promoted to a request.

Project — *Purpose:* the business context that parts/BOMs/orders belong to (e.g., "TVCA"). Also covers internal/R&D and wall-replenishment as project-like buckets. *Relationships:* has Collections, BOMs, Orders. *Ownership:* a lead/designer. *Lifecycle:* active → archived.

BOM — *Purpose:* a validated, sourced bill of materials for a build. *Relationships:* belongs to Project; created by Designer; has many BOMLines; tied to a build quantity/overage; flows to procurement. *Ownership:* designer (origin) → production (processing). *Lifecycle:* received → validated → normalized → sourced → exceptions-resolved → ready-for-purchasing → archived.

BOMLine — *Purpose*: one part + quantity in a BOM, with sourcing decision and status. *Relationships*: belongs to BOM; references Component + SupplierOffer. *Lifecycle*: unsourced → sourced / needs-review / exception → replaced.

ProcurementRequest / PurchaseRequest — *Purpose*: a request from designer or production into purchasing (the "request to order" / "submit to purchasing" object). *Relationships*: references a Collection or BOM; assigned to Purchasing; has Reviews and Comments. *Ownership*: requester → purchasing. *Lifecycle*: submitted → in-review → approved / rejected (with comments) → ordered.

PurchaseOrder — *Purpose*: the actual order placed at a supplier. *Relationships*: derives from a ProcurementRequest; has a supplier, vendor order id, tracking number(s), and a customer reference for attribution. *Ownership*: purchasing. *Lifecycle*: created → ordered → shipped (tracking) → received → closed.

InventoryItem / StockLocation ("the wall") (future) — *Purpose*: physical stock and its location/disposition. *Relationships*: references Component; linked to the PurchaseOrder it arrived on and the owner/project it's attributed to. *Lifecycle*: received → attributed → on-wall / in-project / scrapped → consumed.

Notification — *Purpose*: tell a user that work has moved to them. *Relationships*: targets a User; references the source object (BOM, Request, etc.). *Lifecycle*: created → delivered → read → acted-on.

Review / Approval — *Purpose*: the human checkpoint required by the "checks & balances" principle. *Relationships*: on a Request/BOM/change; has an approver and a decision. *Lifecycle*: requested → approved / rejected (with comments).

Comment — *Purpose*: threaded discussion attached to any object (replaces email/Teams). *Relationships*: polymorphic to BOM/Request/Collection/etc. *Lifecycle*: posted → edited → resolved.

AuditEvent — *Purpose*: immutable record of every state change and action, satisfying the traceability principle. *Relationships*: references actor + object + before/after. *Lifecycle*: append-only.

A consistent metadata envelope on the major objects — owner, creator, project, status, source supplier, created/modified timestamps, assigned user, workflow stage, related records — directly implements the traceability principle from the scope document.

Section 7 — Risk Analysis

Technical risks

- *Supplier API fragility and rate limits.* Mouser throttles (the code already paces requests at ~2.1s and retries); DigiKey's 3-legged OAuth refresh tokens expire and currently require manual re-pasting. *Mitigation:* server-side credential management, async I/O, a robust shared cache with freshness, background jobs with retry/backoff, and graceful degradation (serve cached data, queue refreshes).
- *Error-swallowing.* Several POC paths catch broadly and `print`/continue, conflicting with the "never silently lose data" principle. *Mitigation:* structured logging, explicit error states on objects, and a visible failure/review queue rather than console prints.
- *Replacement-suggestion is genuinely hard.* Parametric matching ("five alternates with the same characteristics") is non-trivial and depends on parametric data quality. *Mitigation:* start by surfacing supplier-provided alternates/substitutes (Nexar/Octopart, DigiKey/Mouser "similar parts"), keep the human as the chooser, and treat full parametric matching as a later iteration.

Product risks

- *Scope sprawl vs. a tiny, part-time team.* The vision is large; the build cadence is ~30-45 min/day with effectively one developer who is primarily a hardware engineer. The dominant risk is building a sprawling platform that never ships. *Mitigation:* ruthless MVP, a thin foundation first, and visible value at every phase.
- *The C#/rewrite temptation.* The founder wondered about rewriting hot paths from Python to C# because "Python is slow." **This is a misdiagnosis worth challenging directly:** the bottleneck is supplier API latency (I/O-bound), not Python CPU time. Async I/O, caching, and background jobs solve it; a C# rewrite would discard the working domain logic and the team's Python familiarity for no real gain at this scale. *Mitigation:* keep Python, fix the I/O architecture.

Workflow risks

- *Handoffs that don't actually unblock people.* If notifications are noisy or status is unreliable, people revert to Teams/email and the platform loses its purpose. *Mitigation:* make status the single source of truth, keep notifications meaningful, and design the handoff UX around "what do I need to act on now."
- *Attribution still failing at receiving.* The customer-reference approach depends on suppliers faithfully round-tripping the field — an open question the team is literally about to test. *Mitigation:* validate the supplier round-trip before depending on it; design barcode scanning as the fallback.

Scalability risks

- *Cache staleness at higher volume.* More BOMs and parts mean more sourcing calls against rate-limited APIs. *Mitigation:* shared cache + dedup + scheduled refresh; this

is already the right instinct in the POC.

- *Single-supplier decision logic.* Won't scale to real procurement optimization or additional suppliers (Newark already appears in the data). *Mitigation:* make the decision policy pluggable.

User-adoption risks

- *Slower than the status quo.* The current direct-to-supplier process "works" for the founder. If the platform is slower or more annoying, it won't be used. *Mitigation:* the single-part search bar and persisted collections must be genuinely faster than opening Mouser/DigiKey; measure this.
- *Trust in automation.* The team explicitly distrusts auto-ordering ("what if it glitches and orders five carts"). *Mitigation:* honor the "automation assists, never decides" principle — keep human approval gates and earn trust before any auto-ordering.
- *Bus factor / code custody.* The codebase lives on a personal GitHub; one developer. *Mitigation:* move to the company GitHub early, document, and keep the architecture conventional enough that another developer could pick it up.

Operational / compliance risk

- *Hosting and data residency are undecided and architecture-affecting.* The team is unsure whether to use cloud (AWS) or an internal server, partly due to possible **contract/confidentiality constraints** on customer BOMs (the repo already gitignores customer BOMs). *Mitigation:* decide data-residency requirements early, and choose a stack that containerizes cleanly so it can deploy to either cloud or on-prem without rework.
-

Section 8 — Product Gaps

What stakeholders want that does not currently exist

- Persistent designer collections (the most-requested feature).
- A single-part search bar.
- Multi-user identity, roles, and role-specific front ends.
- Push/pull handoffs and notifications between departments.
- Order status and tracking visibility.
- Replacement/alternate suggestions.
- Inventory / "the wall," receiving, and order attribution.
- An audit trail / traceability for every object.

What the proof-of-concept fundamentally cannot support (without new foundations)

- Anything stateful or collaborative — there is no database, no users, no notifications, no workflow engine. The POC is architecturally a batch script; the vision requires a system of record. This is not a feature gap, it is a foundation gap.
- Anything multi-user — OAuth and configuration are per-operator and local.
- Anything asynchronous — the pipeline is synchronous and blocking.

Missing requirements that should be clarified before development continues (see Section 11 for the full question list)

- The relationship between PartsBox, the future Inventory module, and "the wall."
 - Whether suppliers reliably round-trip the customer-reference field.
 - The intended supplier policy beyond Mouser/DigiKey (Newark already appears in real data).
 - Overage handling and the build/unit/project quantity model.
 - Hosting and data-residency constraints.
 - Who the actual users are and how authentication will work.
-

Section 9 — Architecture Recommendation

The guiding principle: **preserve the domain logic, replace the delivery mechanism, and build the missing foundations.** Concretely, extract the cleaning/aliasing/decision/supplier logic into a reusable "sourcing core" library, put a real backend and database around it, and replace Streamlit with a role-aware front end.

System architecture — modular, API-driven, service-oriented (not micro-services yet).

A single backend application exposing a clean API, with the domain logic packaged as an internal library and long-running work pushed to background jobs. This satisfies "modular, API-driven, extensible" without the operational overhead of micro-services that a tiny team cannot run. *Reasoning:* the scope document asks for modularity and future ERP/inventory/cloud integration; a well-layered modular monolith gives that extensibility while staying deployable and debuggable by one or two people.

Backend architecture — Python (FastAPI), keep the existing logic. FastAPI keeps the team in the language they already know and in which the valuable logic is written, gives first-class async I/O (the right answer to the supplier-latency bottleneck), and produces an OpenAPI contract the front end and future integrations can build against. *Reasoning:* this directly counters the C#-rewrite temptation — the bottleneck is I/O, not Python — and avoids discarding proven code.

Background processing — a job queue/worker (e.g., RQ/Celery/arq) + Redis. Sourcing, cart/list creation, obsolescence re-checks, and notifications run as jobs with retry/backoff and progress reporting, not inside a web request. *Reasoning:* aligns with the platform principle "prefer asynchronous processing for long-running operations, with clear status indicators," and is the only sane way to handle rate-limited supplier APIs at scale.

Database architecture — PostgreSQL as the system of record. A relational schema modeling the Section 6 objects, with the consistent traceability envelope (owner/creator/project/status/timestamps/stage/related records) and an append-only AuditEvent table. *Reasoning:* the data is highly relational (users ↔ collections ↔ BOMs ↔ requests ↔ orders ↔ inventory), and the traceability and audit requirements demand durable, queryable, consistent storage. The existing JSON lookup cache graduates into Postgres/Redis with freshness timestamps.

Frontend architecture — a role-aware SPA (e.g., React), replacing Streamlit. Role-specific dashboards (Designer / Production / Purchasing / Inventory / Management), an in-app notification feed, and editable persistent views. *Reasoning:* the vision's "role picker / Raul's corner / tailored front ends," persistence, and collaboration cannot be expressed in a single Streamlit toggle screen. The scope document also explicitly prioritizes workflow efficiency over visual polish — so the front end should be fast and task-shaped, not marketing-styled.

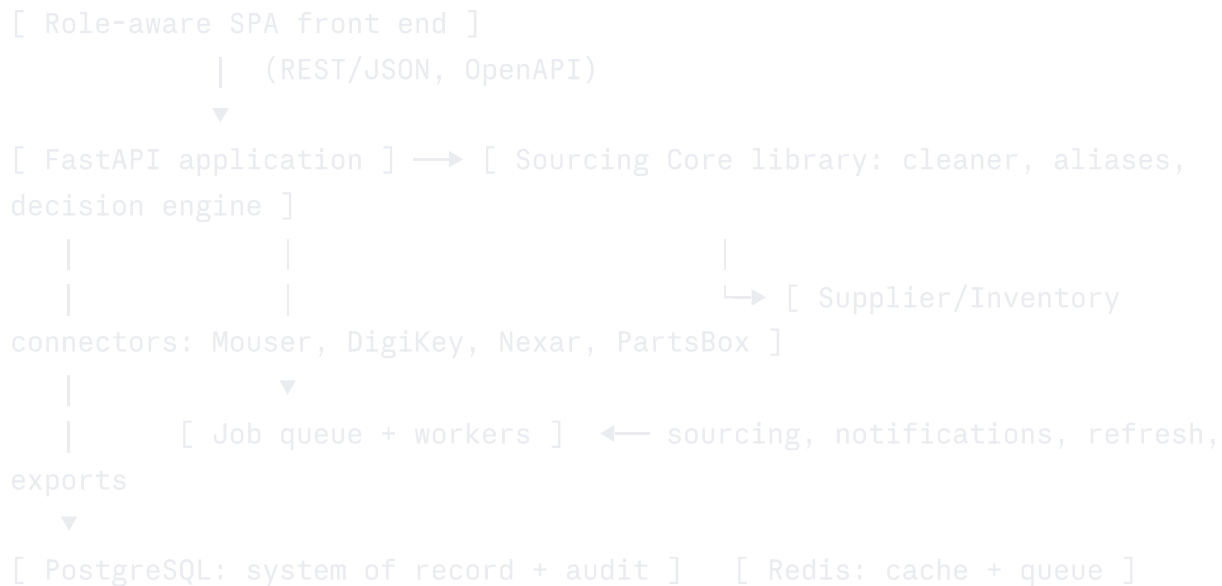
Notification architecture — in-app first, channels later. A Notification object + feed delivered in-app initially; email and Teams as additional delivery channels added behind the same eventing layer once the in-app loop is trusted. *Reasoning:* the founder identified notifications as the hard part and is comfortable with slight latency; starting in-app de-risks it, and an event-driven design lets email/Teams be added without re-plumbing.

Workflow architecture — an explicit state machine per object. Each major object (BOM, Request, Order, Collection) has defined states and allowed transitions; transitions emit AuditEvents and (across person boundaries) Notifications. Human-approval gates are first-class transition guards. *Reasoning:* this is the literal embodiment of the push/pull + checks-and-balances principles, and it keeps "automation assists, never decides" enforceable in code rather than by convention.

Integration architecture — supplier and inventory connectors behind interfaces. Mouser, DigiKey, Nexar/Octopart, and PartsBox sit behind a common connector interface so suppliers (e.g., Newark) and inventory backends can be added without touching the decision engine. Credentials are managed server-side. *Reasoning:* extensibility ("additional suppliers / integrations without major rewrites") and the need to retire per-operator OAuth.

Deployment — containerized, deployment-agnostic. Package the backend, worker, and database as containers so the same artifact runs on AWS or an internal server. *Reasoning:* hosting/data-residency is undecided and possibly contract-constrained; a containerized stack defers that decision without architectural lock-in.

A simple layered view:



Section 10 — Strategic Roadmap

The roadmap is sequenced so that each phase is independently useful and the foundation is built once. It assumes a small, part-time team — so each phase is scoped to ship.

Phase 1 — MVP: Foundation + Designer value

- **Goals:** stand up the system of record; deliver the single most-wanted designer feature; prove the existing logic works behind an API.
- **Features:** Postgres + core objects (User, Role, Component, Collection, Project, SupplierOffer) with auth and roles; the **sourcing core** extracted from the POC into a reusable library behind a FastAPI service; **single-part search bar + check sourcing; persistent designer collections** (create/edit/save, item count + per-supplier breakdown); background sourcing jobs + shared cache; minimal in-app status. Streamlit retired for designers.
- **Dependencies:** hosting/data-residency decision; auth source; migration of the cleaning/decision logic into the library.
- **Risks:** foundation work has no immediately flashy payoff — mitigate by shipping the collection + search feature early so designers feel value fast.

Phase 2 — Internal Production Release: BOM + Purchasing loop

- **Goals:** automate the production BOM pipeline end-to-end and give purchasing a real review/order workflow with visible status.

- **Features:** BOM upload + **validation gate** + normalization (build qty, overage); full sourcing + exception detection; **procurement package** generation; PartsBox project/import wired into the object model; **purchasing review / approve / reject** with comments; **PurchaseOrder** object with **order status** + **tracking ingestion**; **audit trail**; **in-app notifications**; role-aware dashboards replacing the remaining Streamlit UI.
- **Dependencies:** Phase 1 foundation; supplier order/tracking data access.
- **Risks:** ordering stays manual (by design/trust) — make sure status/tracking visibility alone delivers the "stop asking did-you-order-it" win.

Phase 3 — Workflow Platform: Collaboration + Inventory

- **Goals:** make work move between departments without conversation; close the receiving/attribution loop.
- **Features:** **push/pull handoffs** (production↔designer exception loop, designer→purchasing requests) with SLA/timed reminders; **email/Teams** notification channels; **replacement/alternate suggestion** (start with supplier-provided alternates via Nexar/DigiKey/Mouser, human chooses); **Inventory / "the wall"** module with **receiving + attribution** (customer-reference round-trip and/or barcode scanning) and the wall/project/scrap disposition checklist; comment history everywhere.
- **Dependencies:** Phase 2 objects + notifications; supplier round-trip validation; barcode hardware decision.
- **Risks:** replacement matching and attribution are the hardest items here — keep them human-in-the-loop and iterative; do not over-promise full parametric matching.

Phase 4 — Advanced Automation: Trust-earned automation + insight

- **Goals:** automate the actions the team now does manually, but only after trust is earned; add oversight and reporting.
- **Features:** **opt-in automated ordering** with strict guardrails and approval gates; **analytics** (spend, cycle time, exception rate, supplier mix, on-time receiving); **multi-supplier / split-order** optimization (add Newark and others); **ERP / multi-location inventory / mobile** as demand appears.
- **Dependencies:** a track record of reliable manual ordering and accurate status; management requirements for reporting.
- **Risks:** automated ordering is the highest-trust, highest-blast-radius feature — gate it heavily, keep it reversible, and never make it the default.

Section 11 — Unanswered Questions & Assumptions to Resolve

Questions that should be answered before development continues

1. **Hosting & data residency:** cloud (AWS) or internal server? Are there contractual/confidentiality constraints on customer BOMs that forbid third-party hosting? This affects deployment but should not affect the (containerized) architecture.
2. **Users & identity:** who are the actual users, how many, and how will they authenticate (company SSO, Google Workspace, simple internal accounts)?
3. **PartsBox's role:** is PartsBox the long-term inventory system of record, a stopgap, or to be replaced by the future Inventory module? How does it relate to "the wall"?
4. **Customer-reference round-trip:** do Mouser/DigiKey reliably echo back a customer reference field on orders/packing data? (The team was about to test this — its outcome decides the near-term attribution approach vs. barcode scanning.)
5. **Supplier policy:** which suppliers beyond Mouser/DigiKey must be supported? Newark already appears in real BOM data. What are the rules for choosing among them (price, lead time, stock, preferred vendor)?
6. **Quantity model:** confirm the build/unit/project quantity model and whether an **overage** % must be applied (the production diagram says yes; the code does not implement it).
7. **Replacement scope:** how sophisticated must alternate suggestions be — supplier-provided "similar parts," or true parametric matching on characteristics? The latter is a major effort.
8. **Notifications:** is in-app sufficient for launch, with email/Teams later, or is a channel required from day one? What latency is acceptable? (Conversation suggests slight latency is fine.)
9. **Auto-ordering:** confirm that automated ordering stays out of scope until explicitly trusted, consistent with the "automation assists, never decides" principle.
10. **Code custody:** move the repository from the personal GitHub to the company GitHub; confirm access and whether a company email is required.

Assumptions made in this report (please confirm or correct)

- The team is small and part-time, with effectively one developer who is primarily a hardware/design engineer; the dominant risk is shipping at all, so MVP scope must stay tight.
- The valuable, hard-won logic is the cleaning/normalization, manufacturer-alias/MPN equivalence, and the supplier decision engine — these are worth preserving and refactoring rather than rewriting.
- Python is the right language to continue in; the performance concern is I/O-bound (supplier APIs), not CPU-bound, so a C# rewrite is not warranted at this stage.
- "The wall" is internal component stock, distinct from per-project parts, and parts can move from R&D → wall → project → scrap via a disposition decision.

- Concurrency is low (well under hundreds of simultaneous users) for the foreseeable future, which keeps a modular monolith appropriate.
 - Confidential customer BOMs must not leak (the repo's gitignore behavior implies this is already a concern), which raises the priority of the hosting/data-residency decision.
-

End of discovery report. No implementation work has been started and no code has been generated, per the engagement scope. The recommended next step is a working session to resolve the Section 11 questions, followed by low-fidelity mockups of the Designer and Purchasing flows (the two highest-value, lowest-risk surfaces) to confirm the workflow before any build begins.